QUALITY ASSURANCE · TEST PLAN

TaskFlow

Test plan for the task management web application

Plan version 1.0 | Status: Draft | Source PRD: Version 1.0 · Draft | Prepared: 23 October 2026

Planning note — requirements are still draft

This plan is based only on the supplied TaskFlow PRD. The PRD identifies several sample targets that require validation, and leaves environment URLs, test dates, ownership details, and some product behavior open. Such items are marked TBD or listed for confirmation; they are not assumed.

Contents

1. Objective
2. Scope
3. Inclusions & coverage
4. Test environments & data
5. Defect reporting
6. Test strategy
7. Test schedule
8. Test deliverables
9. Entry & exit criteria
10. Test execution
11. Test closure
12. Tools
13. Risks & mitigations
14. Approvals
15. Requirement coverage
16. Open questions

1. Objective

Validate that TaskFlow lets individual users register, authenticate, manage and organize their tasks while protecting user data and meeting the performance and reliability expectations that are agreed for release.

Testing will verify the functional requirements FR-01 through FR-07, the stated non-functional requirements, the API response examples and the QA expectations in PRD version 1.0. The product is a web application for personal task management; team collaboration, notifications, attachments, calendar integration, third-party login, AI recommendations and mobile applications are explicitly out of scope.

2. Scope

In scope

- User registration, login, logout, validation, and protected access.
- Task create, list, retrieve, update, and delete behavior.
- Task title, description, priority, status, due date, ownership, and filters.
- Input validation, error responses, user-level authorization, and data isolation.
- API, UI, unit, integration, functional, negative, security-control, performance, smoke, and regression testing.
- Database persistence and API response/schema checks, as defined by the final contracts.

Out of scope

- Team collaboration and assigning tasks to other users.
- Email/SMS notifications, file attachments, and calendar integration.
- Third-party login, AI recommendations, and native mobile applications.
- Unspecified features such as password reset, recurring schedules, task archiving, and deletion retention policy until requirements are decided.

3. Inclusions & coverage

AREA	PLANNED VERIFICATION	SOURCE
Registration	Valid account creation; required name, email and password validation; malformed email; duplicate email; password policy; password hashing at rest.	FR-01
Authentication	Valid and invalid credentials; missing credentials; authenticated and unauthenticated access; logout invalidation according to the approved session/token design; prevent access to another user's data.	FR-02
Create task	Valid task creation; title required and 3–100 characters; description optional and up to 1,000 characters; accepted priorities Low, Medium, High; ownership cannot be forged in request data.	FR-03
View tasks	List and retrieve tasks; title, priority, status and due date; consistent ordering; empty state; ownership isolation; invalid pagination handling once contract behavior is defined.	FR-04
Update task	Valid/invalid field updates; supported statuses Pending, In Progress, Completed; nonexistent task response; ownership check; persisted values shown on retrieval.	FR-05
Delete task	Authorized deletion; nonexistent task response; ownership check; deleted task removed from active list and unavailable through normal task API.	FR-06
Filter tasks	Status and priority filters; due-date filter; combined filters match all conditions; invalid filter values rejected; no cross-user disclosure.	FR-07
Validation & errors	Malformed JSON, missing fields, boundary values, invalid types/enums, consistent error format and appropriate status codes against the finalized API contract.	FR-01–FR-07; API requirements

AREA	PLANNED VERIFICATION	SOURCE
Security	Server-side validation, HTTPS for production traffic, authentication and authorization enforcement, secure password storage, secrets handling, and sensitive-log review.	NFR 6.2
Performance & reliability	Measure agreed API p95 target under defined load; expected concurrency; graceful/consistent errors; operational error logging; atomic database operations where applicable.	NFR 6.1, 6.3
Automation & regression	Unit, integration, API, UI, and regression suites; verify pipeline checks as the deployment approach is finalized.	QA requirements; proposed architecture

Scope boundary: The proposed React, Node.js/TypeScript, AWS Lambda, PostgreSQL, Zod, CloudFormation, and Jenkins architecture is a sample proposal, not a mandatory product requirement. Test implementation details should be aligned after the technical design is approved.

4. Test environments & data

ENVIRONMENT / ITEM	PLAN	STATUS
QA / test environment	Use a deployed TaskFlow environment with required APIs, UI, database, and authentication integration available. Base URL and access details are not specified.	TBD
Staging	Release plan calls for deployment to staging, smoke testing, and release approval. URL, data policy, and access are not specified.	TBD
Production	Production verification is limited to approved release/deployment checks and monitoring; production test permissions and data safeguards must be agreed.	TBD

ENVIRONMENT / ITEM	PLAN	STATUS
Platform	Target is a web application. Supported browser names/versions, operating systems, devices, and viewport matrix are not defined in the PRD.	Confirm
Test accounts	Prepare distinct users to validate ownership boundaries, account registration/login, empty lists, and task isolation. Use approved non-production data.	Required
Task data	Include valid and boundary task titles (3 and 100 characters), optional description through 1,000 characters, supported and unsupported priority/status values, due dates, and tasks for separate owners.	Required
Load profile	Expected concurrent users, request volumes, test duration, and workload mix are not given; define and approve before performance execution.	TBD

5. Defect reporting procedure

1. Record each observed deviation from the approved PRD, acceptance criteria, or finalized API/UI contract with a concise title, affected requirement, environment/build, preconditions, exact reproduction steps, expected and actual results, and supporting evidence where safe.
2. Include request/response details for API issues, excluding credentials, tokens, secrets, and sensitive user data. For data-related issues, describe the test record without exposing real personal information.
3. Classify severity and priority using the project's agreed defect taxonomy. The taxonomy, tracking system, triage attendees, response targets, and escalation path are not specified and remain TBD.
4. Track status through triage, fix, retest, and closure. Link the defect to its requirement/test and add a regression check for confirmed fixes where appropriate.
5. Communicate the execution summary, open defects, blocked coverage, and release-impacting risks to the designated QA and project stakeholders.

Tracking tool and roles: TBD. The PRD does not name a defect-management tool or defect owners.

6. Test strategy

6.1 Test levels and types

- **Unit:** Verify focused validation, business rules, and service logic as implemented.
- **Integration:** Verify interactions among authentication, task APIs, persistence, and database constraints.
- **API:** Verify endpoints, access controls, request validation, status codes, response schemas, filtering, and persisted changes.
- **UI:** Verify core user journeys and visible task states on the agreed web/browser matrix.
- **Functional and negative:** Cover acceptance criteria, invalid/missing input, unauthenticated requests, cross-user access, nonexistent records, malformed JSON, and boundary values.
- **Security controls:** Verify the explicit PRD controls before release. Detailed security test scope and authorization/session contract must be agreed.
- **Performance:** Measure API response-time target and concurrency after the workload profile and infrastructure are agreed.
- **Smoke, sanity, retest, and regression:** Run smoke checks on each candidate build, retest fixes, and run the agreed regression suite before release.

6.2 Test design

Use equivalence partitioning and boundary value analysis for field constraints and pagination; decision tables for combined status/priority/due-date filters and authorization outcomes; state-transition checks for task statuses and authentication/logout behavior; use-case and exploratory testing for end-to-end user workflows and error handling. These are test design approaches, not additional product requirements.

6.3 End-to-end flows

1. Register a user, log in, create a task, view it, update fields/status, filter it, delete it, and verify it is absent from the active list.
2. Authenticate as one user and attempt to access, change, or delete another user's task; verify no unauthorized data exposure or mutation.
3. Exercise a new user with no tasks and verify the defined empty state.
4. Submit invalid registration, login, task, filter, pagination, and malformed request data; verify the finalized error contract.

6.4 Success measures

Successful registration	**≥95%**
Proposed sample target; validate the denominator, measurement window, and infrastructure before sign-off.	
Successful task creation	**≥98%**
Proposed sample target; validate against agreed test data and load conditions.	
API response time	**<500 ms**
95th percentile under the agreed test load. Load profile and measurement method are TBD.	
Application availability	**99.5%**
Monthly target; operational measurement and monitoring source are TBD.	

The PRD also sets a release goal of zero known critical security issues. Registration/task creation and performance targets are explicitly described as proposed sample targets requiring validation.

7. Test schedule

The PRD defines planning/design, development, testing, and deployment phases, but provides no calendar dates, sprint duration, or task estimates. Dates and owners are therefore TBD.

Phase 1 · Planning and design

Review/finalize requirements and API contracts; agree acceptance criteria, database schema, environment access, test data, and unresolved policy decisions. QA prepares this plan and derives scenarios.

Dates: TBD

Phase 2 · Development

Development work proceeds; QA performs shift-left review of requirements and testability. Test automation and integration points are aligned with the approved design.

Dates: TBD

Phase 3 · Testing

Run smoke, functional, API, integration, UI, negative, security-control, performance, retest, and regression activities as the required build and dependencies become available.

Dates: TBD

Phase 4 · Deployment

Deploy to staging, complete smoke testing, obtain release approval, deploy to production, and monitor after release.

Dates: TBD

Test cycle duration, execution resource allocation, reporting cadence, and release cutoff are to be agreed with the project team.

8. Test deliverables

- This test plan and its approved revisions.
- Requirement-linked test scenarios and test cases (including API/UI automation candidates).
- Test data and environment readiness notes, without secrets or sensitive data.
- Automation results for unit, integration, API, UI, and regression tests as available.
- Defect records, retest outcomes, and traceability updates.
- Per-cycle execution summary: pass/fail/blocked counts, coverage, defects by agreed severity, risks, and open questions.
- Final test summary and QA release recommendation.

9. Entry & exit criteria

Test planning / design entry

- Requirements and acceptance criteria are available for review.
- Product, technical, and QA stakeholders can clarify open questions.

Exit

- Requirements are reviewed and understood; testable criteria and unresolved decisions are documented.
- Test scenarios, coverage, and dependencies are reviewed.

Test execution entry

- Requirements reviewed; environment available; required APIs deployed; test data available.
- Test scenarios/cases are reviewed or signed off as required by the project.
- Initial build passes smoke testing.

Exit

- All critical test cases pass.
- No unresolved critical or high-severity defects remain according to the approved release policy.
- Agreed regression tests pass; API performance targets are met.
- Results are documented and QA provides a release recommendation.

The PRD contains the entry/exit conditions above. The “high severity” threshold and the project release policy must be formally agreed; otherwise this gate cannot be assessed consistently.

10. Test execution

1. Confirm the build, deployment, requirements baseline, API contract, environment, test accounts, and test data.
2. Run smoke tests for registration/login, protected access, core task creation/retrieval, and service availability. If smoke testing fails, record blockers and defer broad execution until a stable build is available.
3. Execute prioritized functional and negative cases across the FR-01 to FR-07 coverage matrix; record results against the requirement and build.
4. Run integration, security-control, performance, UI, and regression checks when dependencies, approved scope, and environment are ready.
5. Log defects, retest fixes, run impacted regression checks, and communicate status and risk to stakeholders.
6. Repeat test cycles until exit criteria are met or a release decision is made with documented risk acceptance.

Performance gate: Do not report a pass/fail against the 500 ms p95 target until the agreed load profile, environment capacity, measurement window, and response-time sampling approach have been documented.

11. Test closure

Closure entry

- Execution results, defect status, regression results, and performance results (when applicable) are available.
- Release criteria and any residual risks are reviewed with stakeholders.

Closure exit

- Test summary report and QA release recommendation are delivered.
- Coverage, blocked tests, unresolved defects, and accepted risks are documented.
- Required approvals are recorded before the next release phase.

12. Tools

PURPOSE	TOOL / APPROACH IDENTIFIED IN PRD	NOTES
API testing	Postman or Playwright API testing	Choose and confirm project standard.
UI automation	Playwright with JavaScript/TypeScript	Named as proposed architecture; confirm with implementation design.
CI/CD	Jenkins	Named in proposed architecture; pipeline availability and gates to confirm.
Defect tracking	TBD	No defect tracking product is specified in the PRD.
Reporting / evidence	TBD	Agree storage and reporting location before execution.

13. Risks & mitigations

RISK	IMPACT	MITIGATION / RESPONSE
Database connection failures	Users cannot manage tasks.	Verify connection behavior, timeouts, monitoring, and approved retry policies; these operational controls require implementation confirmation.
Unauthorized data access	User data exposure.	Test ownership checks across list, retrieve, update, delete, and request manipulation scenarios.
Slow API responses	Poor user experience and unmet performance goal.	Agree load profile; measure p95; investigate query and service behavior; PRD notes appropriate indexes for common task queries.
Incomplete validation	Invalid or inconsistent data.	Test server-side validation, boundaries, enums, malformed input, and consistent error responses.
Deployment failures	Service interruption.	Verify deployment pipeline checks, staging smoke tests, and rollback procedures before release.
Unresolved PRD open questions	Ambiguous expected behavior and blocked or inconsistent test decisions.	Resolve authentication/token policy, pagination, deletion behavior, notifications, recurring tasks, archive behavior, and expected load before the affected coverage or release gate.
Environment, API, or test-data unavailable	Testing delayed or coverage blocked.	Track readiness as a dependency; identify blocked tests and resume once access, deployment, and safe test data are available.

14. Approvals

ROLE	NAME	STATUS	DATE
Product Owner	TBD	Pending	TBD
Technical Lead	TBD	Pending	TBD
QA Lead	TBD	Pending	TBD
Project Manager	TBD	Pending	TBD

Approval of this plan confirms the scope, test approach, environment and release gates. The PRD lists these approver roles with names TBD and approval status Pending.

15. Requirement coverage map

REQUIREMENT	PRIMARY TEST COVERAGE	PRIORITY
FR-01 User Registration	Valid/invalid registration, uniqueness, required fields, email/password rules, secure password storage.	High — core account entry
FR-02 User Login	Credential checks, missing inputs, protected API access, logout invalidation, cross-user access control.	High — access control
FR-03 Create Task	Valid creation, title/description boundaries, priority enum, owner association and request tampering.	High — core task flow
FR-04 View Tasks	List/detail data, ordering, empty state, ownership isolation, pagination validation.	High — core task flow
FR-05 Update Task	Allowed fields/statuses, invalid values, nonexistent ID, ownership, persisted update.	High — core task flow
FR-06 Delete Task	Authorized delete, nonexistent ID, ownership, active-list removal, normal API retrieval after delete.	High — core task flow

REQUIREMENT	PRIMARY TEST COVERAGE	PRIORITY
FR-07 Filter Tasks	Status, priority, due date, combined conditions, invalid filters, owner isolation.	High — task organization
NFR 6.1 Performance	p95 response time under agreed test load and expected concurrent users.	High — target needs validation
NFR 6.2 Security	Password storage, HTTPS, authentication/authorization, server-side validation, secrets, sensitive logs.	High — release control
NFR 6.3 Reliability / maintainability	Consistent errors, relevant operational logs, atomicity where applicable, automated test and review evidence.	Medium / High — finalize release interpretation

Priorities above reflect the PRD's High product priority, core user goals, and stated release/security gates; no separate project severity or test prioritization scale is supplied.

16. Open questions & decisions needed

Resolve these PRD questions before finalizing test cases, API expectations, or release gates:

- Should password reset be supported?
- Should tasks support recurring schedules or completed-task archiving?
- Should due-date notifications be supported? (Notifications are otherwise listed as out of scope.)
- Should task lists support pagination, and what are the accepted pagination parameters and invalid-parameter behavior?
- Should deletion be soft or permanent?
- What are expected concurrent users and request volumes?
- What are the exact authentication and token-expiration policies, including logout invalidation?
- What are the final API schemas, error codes, and authentication behavior?
- Which QA/staging environments, supported browsers/versions, dates, and defect-management/reporting tools will be used?
- How will sample success/availability/performance targets be measured and accepted?

Release dependency: Until these decisions are recorded, affected test expectations remain conditional and should not be silently treated as settled acceptance criteria.